PrepLang: a data-aware compiler for data-preparation pipelines that reuses its proofs on new batches

Nikhil V

School of Computer Science and Engineering, Vellore Institute of Technology, Vellore 632014, Tamil Nadu, India

Abstract

A data-preparation pipeline is compiled when its input already exists, and then run again on later batches the compiler never saw. PrepLang, a compiler for a small language of table and image pipelines written in C++17, uses both facts. It reads the input at compile time (every CSV cell, Parquet footers, image headers), proves or refutes the pipeline's data contracts before any data is processed, and removes the checks and operations the data makes useless. Every removal stays in the generated pandas and OpenCV program behind a guard that holds the facts about the input it depends on, so the program runs unchanged on new batches and runs a removal again only on a batch that breaks those facts, with the result a fresh compilation would give. The repository holds 299 unit tests with sanitizer, coverage and fuzzing jobs, Lean 4 proofs for a core of the analysis, and one script that reproduces every number in this article.

Keywords: domain-specific language, data-aware compilation, abstract interpretation, speculative optimization, deoptimization, data validation

1. Motivation and significance

Data-preparation mistakes propagate silently into models [1]. A preparation pipeline is usually a script over pandas [2] and OpenCV [3] that cannot know, until it runs, whether a column it normalizes holds numbers or whether the images it saves as PNG hold float pixels; validators such as TFDV [4], Deequ [5], Great Expectations [6] and pandera [7] check the data separately from the code that transforms it. PrepLang uses a property of batch preparation that compilers leave unused: the inputs exist when the pipeline is

Email address: atnikhil06@gmail.com (Nikhil V)

Nr.	Code metadata description	Please fill in this column
C1	Current code version	[v2.0.0: tag this release]
C2	Permanent link to code/repository used for this code version	`https://github.com/Nikhi00718/preLang` [make it public]
C3	Permanent link to Reproducible Capsule	[Docker image ghcr.io/nikhi00718/preplang:2.0.0, from the release workflow]; `eval/reproduce.sh check` builds, tests and regenerates every number of this article from the committed results
C4	Legal Code License	MIT
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	C++17 (no external libraries); Python 3 with pandas, NumPy, PyArrow, OpenCV; Lean 4; CMake
C7	Compilation requirements, operating environments and dependencies	CMake 3.16+, a C++17 compiler (tested in CI: Visual Studio 2022 on Windows, GCC on Ubuntu and macOS); Python 3.12+ with pandas (2.0+, 3.0 tested), PyArrow, OpenCV; Lean 4.34.1 (optional, for the proofs)
C8	If available, link to developer documentation/manual	`README.md` (user manual), `LANGUAGE.md` (reference), `ARCHITECTURE.md`, `CONTRIBUTING.md`; the READMEs of `eval/`, `paper/`, `proofs/`, `docs/`
C9	Support email for questions	atnikhil06@gmail.com

Table 1: Code metadata (mandatory)

compiled. As F# type providers do for types [8], but for values, it reads every CSV cell, Parquet footer and image header, follows types, missing values, value ranges and image properties through the pipeline, proves or refutes each contract before any data is processed, and removes the checks and operations the data makes useless.

Such proofs hold for one file, while the pipeline runs again next month on a file with the same layout and other values. PrepLang therefore computes, for every removal, the facts about the input it depends on (its *item certificate*), and the generated program checks them on each new batch: a removal whose facts hold stays removed, one whose facts a batch breaks runs again (*selective deoptimization*, from dynamic compilers [9, 10]), and a batch is refused only for facts no run-time code can stand in for, such as a changed column type. A user writes a `.prep` program, compiles it against the current batch, reads the diagnostics and deploys the generated program for every later batch. The theory (with its Lean proofs) and the full evaluation are documented in the repository (`paper/`, `docs/`); this article describes the software.

Table 2 places PrepLang among the closest tools. Validators know the values but check them apart from the code and again on every batch; compile-time analyses follow the code but know only declared or inferred structure. To our knowledge, no other tool both decides value contracts at program points from the real data and carries those decisions safely to later batches.

Table 2: PrepLang and the closest tools. *Values*: missing values and value ranges.

Tool	Decides at	Knows about the data	Follows code	On a new batch
TFDV [4], Deequ [5], GX [6], pandera [7]	run time	values, from each batch	no	validates it again
F# type providers [8]	compile time	types, from a sample	no	–
Zhuang and Lu [11]	compile time	column types, from the code	yes	–
Gerstl et al. [12]	compile time	R data-frame shapes, optionally from the data	yes	–
Mirji [13]	compile time	declared schemas	yes	checks the output schema
PrepLang	**compile time**	**types and values of every cell; image headers**	**yes**	**admits by certificate; reruns only the removals it breaks**

2. Software description

2.1. Software architecture

The compiler (`src/`, about 11,800 lines of C++17 without external libraries) runs lexer, LL(1) parser, semantic analysis, IR, optimizer, certifier and code generator in turn. The *schema* readers scan CSV files in parallel and read Parquet footers and PNG and JPEG headers; the *semantic analyzer* computes the abstract dataset state (types, missing values, value intervals, image properties) and decides every contract; the *optimizer* removes, speculatively, what the state shows to have no effect; the *certifier* re-runs the analysis under weakened input facts, reading no data, to find the admission certificate and one item certificate per removal (Fig. 1); the *back end* generates one Python program that admits a batch and guards every removed instruction with `deopt(i)`, and writes the certificates to `preplang_contract.json`.

The repository also holds `proofs/` (1477 lines of Lean 4 [14]: selective deoptimization is exact for any monotone and sound analysis, and the analysis of a core of the language is sound and monotone), `tests/` (299 checks in 10 suites, with randomized soundness tests), a lifter of pandas scripts (`frontend/`), the composition and release tools (`tools/`), a local editor app (`studio/`) and the evaluation (`eval/`).

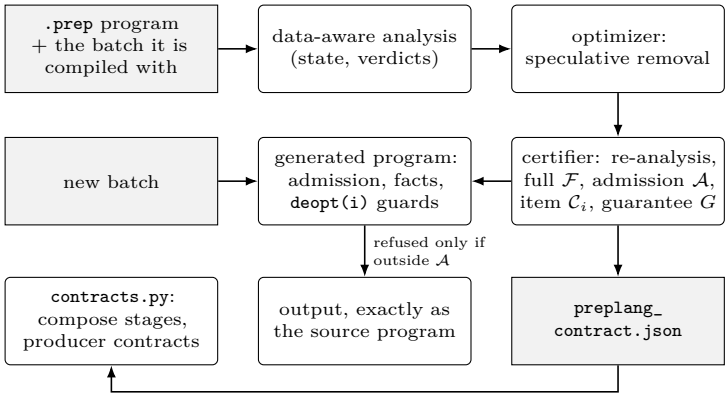


Figure 1: How PrepLang compiles once and reuses the result. The certifier re-runs the analysis with weaker input facts, reading no data; on a new batch, the generated program runs a removed instruction i only if the batch breaks its item certificate $\mathcal{C}_i$.

Continuous integration builds and tests the compiler on Windows (MSVC), Ubuntu and macOS (GCC), compiles every example end to end, and runs the tests under AddressSanitizer and UndefinedBehaviorSanitizer, a line-coverage gate, `clang-format` and `clang-tidy`, and libFuzzer on the three parts that read untrusted bytes (the `.prep` front end, the CSV scanner and the Parquet footer reader); it also checks the Lean proofs and builds the Docker image. Each release publishes compilers for the three systems that need no C++ toolchain. `LANGUAGE.md` is the reference of the language, the command line and the generated program.

2.2. Software functionalities

- **Compile-time checking:** misspelled or removed columns, type errors and broken contracts (`NOT_NULL`, `BETWEEN`, `UNIQUE`, `REFERENCES`, schemas, image channels and sizes, match rates of images and labels), lookup joins on keys of other types or with repeated values, reported with the line, the offending row and a hint; groups (`AGGREGATE`) keep their facts.
- **Data-driven optimization:** proven checks and no-effect operations are removed, unused columns are not loaded, image pipelines are streamed; rewrites that change the output need `--approx`.
- **Reuse:** admission by certificate, and re-enabled removals with the reason printed.
- **Beyond one program:** `tools/contracts.py` checks one compiled stage's guarantees against the next stage's certificates; `frontend/pandas2prep.py` lifts the cleaning part of pandas scripts into PrepLang; and with `--facts` a program compiles from differentially private facts

its data owner released (`tools/release_facts.py`), opening no input file, while every run stays exact.

- **Inspection:** every phase can be printed (`--tokens` to `--python`), with an execution plan (`--explain`) and the files a run would touch (`--dry-run`).

3. Illustrative example

```
LOAD "students.csv" AS students;
EXPECT students.age NUMBER;
EXPECT students.marks BETWEEN 0 AND 100;
EXPECT students.name NOT_NULL;
FILTER students WHERE marks >= 50;
EXPECT students.marks BETWEEN 60 AND 100;
SAVE students AS "clean.csv";
```

Compiled against an eight-row `students.csv` (from `examples/tabular/`), PrepLang proves the first three contracts and checks the fourth at run time, since after the filter it only knows that the marks lie in $[50, 95]$. It removes the range and not-null checks speculatively, and `--certificate` shows the facts the range check needs: marks at least 0 and at most 100. Run on a later batch in which a mark is 104, the generated program re-enables the check and stops where a fresh compilation would:

```
Deoptimized line 3: EXPECT students.marks BETWEEN 0 AND 100:
  marks goes up to 104, above 100
ContractError: line 3: students.marks has values outside 0 to 100
```

A batch within the facts runs with both checks removed; a batch whose `age` holds text is refused, since no generated code can stand in for the analysis of another type.

The same analysis covers images, from their headers alone. In `examples/images/linked.prep`, images and their labels are linked and checked, resized, normalized to $[0, 1]$ and saved as NumPy arrays. Changing the format to PNG gives, before any pixel is decoded:

```
error[type]: cannot save FLOAT32 pixels as PNG:
             NORMALIZE at line 10 scaled them to [0, 1]
   --> examples/images/linked.prep:13:1
13 | SAVE_IMAGES imgs TO "processed/ml_ready/";
   = hint: image files store 8-bit pixels, so these would come out
     almost black; add CONVERT imgs TO NPY; to keep the float
     values
```

The equivalent hand-written OpenCV script runs without an error and writes almost black images.

4. Impact

The evaluation in `eval/` answers six questions (`eval/evaluate.py` writes the answers with every number below):

- **Correctness: is the output right?** 2,000 random programs (with joins and groups), compiled with and without the optimizer, agree cell by cell with a reference interpreter that shares no code with the compiler, and reused on a second batch none of the 1,535 runs differs from it; before two fixes this test led to (a pruning bug, a pandas 3 incompatibility), 109 optimized runs were wrong. Three more seeds (6,000 programs) found one more bug (a statistic fill removed as a dead store, though it can stop the program; 1 wrong run), fixed with a regression test, after which all 6,000 agree and none of 4,646 reuses is wrong. Reused on new batches, 64,179 runs after 4,879 private compilations and every run of the experiments below did what a fresh compilation does.
- **Performance.** An optimized table pipeline runs in 0.47 s instead of 0.79 s after Python start-up, and an image pipeline as fast as a hand-written streaming script, in 10 MiB instead of the 922 MiB of a script that loads every image.
- **Optimization effectiveness.** On 436 examples and lifted programs the optimizer removes 14% of the instructions, 75% of the run-time checks and 2% of the columns read, with identical output.
- **Practical usefulness.** 18 of 44 real failures from GitHub issues are reported before any data is processed; lifted pandas scripts from GitHub reproduce their results on new batches in 3198 of 3212 runs, and in all 3255 monthly runs of 72 scripts written by others; on 16 million Parquet rows a misspelled column is reported in 0.02 s instead of after 2.81 s.
- **Comparison.** On 238 real monthly batches the compiled programs handle 221 (93%) as a fresh compilation would, with 0 wrong; Great Expectations and pandera refuse 92–93% of the months a fresh compilation runs, and TFDV admits months on which the program then differs. All 24 injected mistakes are reported at compile time, where a script-style program crashes on 20 and writes wrong output on 4. From private facts at $\varepsilon = 1$ the taxi and traffic pipelines keep every removal (fixed bins: 60%, 83%); HDMM [15] and AIM [16] do no better.
- **Overhead.** Compiling takes a median of 28 ms and at most 8 MiB (65 MiB for a million CSV rows, 6 MiB for a 346 MiB Parquet file, of which only the footer is read); admitting a new batch changes run time by -1%.

`eval/reproduce.sh check` rebuilds the compiler, runs every test and regenerates every number above from the committed results, without network access; `full` reruns the experiments and `baselines` the validators and differential-privacy mechanisms, in pinned environments (`eval/README.md`). The evaluation is reusable on its own: the differential harness with its reference interpreter tests any change to the optimizer, and the GitHub script corpora are frozen as evaluated, each file by its URL at a fixed commit and its hash (1,773 of the 1,778 files could still be downloaded on 23 October 2026), so other validators and pipeline analyses can be compared on the same programs and batches. PrepLang is a research prototype by one author, begun in a compiler-design course; it has no users outside the author's experiments yet. Its abstract state, certifier and code generator are separate modules that others can extend.

5. Limitations and conclusions

Joins are lookups on a key with one name in both tables (no many-to-many joins), groups have five aggregates, and only Python is generated; a changed column type is refused rather than handled; the mechanized proofs cover a core of the analysis without joins and groups, and the rest is tested; the compiler is tested on Windows (MSVC), Ubuntu and macOS (GCC). Planned work is general joins, generated code that adapts to type changes, and a Python package. PrepLang compiles a pipeline once and runs it on later batches exactly as a fresh compilation would, re-enabling only the removals a batch makes unsafe.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

[1] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 2503–2511.

[2] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf. (SciPy)*, 2010, pp. 56–61.

[3] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.

[4] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in *Proc. Conf. on Machine Learning and Systems (MLSys)*, 2019.

[5] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," *Proc. VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, 2018.

[6] Great Expectations, "Great Expectations: data quality testing," `https://greatexpectations.io`.

[7] N. Bantilan, "pandera: Statistical data validation of pandas dataframes," in *Proc. 19th Python in Science Conf. (SciPy)*, 2020, pp. 116–124.

[8] T. Petricek, G. Guerra, and D. Syme, "Types from data: Making structured data first-class citizens in F#," in *Proc. 37th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2016, pp. 477–490.

[9] U. Hölzle, C. Chambers, and D. Ungar, "Debugging optimized code with dynamic deoptimization," in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 1992, pp. 32–43.

[10] A. Barrière, S. Blazy, O. Flückiger, D. Pichardie, and J. Vitek, "Formally verified speculation and deoptimization in a JIT compiler," *Proc. ACM Program. Lang.*, vol. 5, no. POPL, pp. 1–26, 2021.

[11] Y. Y. Zhuang and M.-Y. Lu, "Enabling type checking on columns in data frame libraries by abstract interpretation," *IEEE Access*, vol. 10, 2022.

[12] O. Gerstl, F. Sihler, and M. Tichy, "Inferring the shape of data frames in R programs using abstract interpretation," arXiv:2607.03889, 2026.

[13] V. Mirji, "Shift schema drift left: Policy-aware compile-time contracts for typed JVM and Spark pipelines," arXiv:2604.16986, 2026.

[14] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Proc. 28th Int. Conf. on Automated Deduction (CADE)*, LNCS 12699, 2021, pp. 625–635.

[15] R. McKenna, G. Miklau, M. Hay, and A. Machanavajjhala, "Optimizing error of high-dimensional statistical queries under differential privacy," *Proc. VLDB Endowment*, vol. 11, no. 10, pp. 1206–1219, 2018.

[16] R. McKenna, B. Mullins, D. Sheldon, and G. Miklau, "AIM: An adaptive and iterative mechanism for differentially private synthetic data," *Proc. VLDB Endowment*, vol. 15, no. 11, pp. 2599–2612, 2022.